Project: Unitree G1 Soccer Skills

CS2810 Teaching Team

1 Overview

This is the project component of Embodied AI CS2810 (Spring 2026). The task is to develop reinforcement learning skills for humanoid robot soccer using the Unitree G1 (29 dof) platform in `mjlab` physics simulation. The project centers on two tasks:

- **Shooting**: The G1 robot must approach and kick a stationary ball toward a goal. This is a perception-guided motor skill requiring motion tracking priors and ball trajectory prediction.
- **Goalkeeping**: The G1 robot must intercept an incoming ball launched along a parabolic trajectory. This is a reactive interception task requiring rapid end-effector positioning and ball trajectory estimation.

2 Project Phases & Grading Rubric

The project is evaluated out of 100 total points, broken down into three primary components: Phase 1 (60%), Phase 2 (30% = 15% Shooting + 15% Goalkeeping), and Submission Deliverables (10%).

2.1 Phase 1: Basic Shooting and Goalkeeping (60% of project score)

Implement single-agent training for individual shooter and goalkeeper tasks in a non-adversarial setting. You may refer to [1] and [2] for guidance on environment design, reward shaping, and training strategies. This phase evaluates the fundamental motor control and perception integration of your policies.

The 60% allocation is divided equally between the two tasks (30% each), using a strict tiered accuracy model calculated over 50 evaluation trials:

- **Shooter Policy (30 points total):** Evaluated using `eval_naive_shooter.py`. Success is defined as the ball completely crossing the goal plane inside the frame posts.
- **Goalkeeper Policy (30 points total):** Evaluated using `eval_naive_goalkeeper.py`. Success is defined as a clean block where the ball is intercepted or deflected outside the goal frame.

Both tasks are scored with a linear formula applied to the success/block rate over 50 evaluation trials:

$$\text{score} = \max\left(0, \frac{\text{success\_rate} - 0.8}{0.2}\right) \times 30$$

Success rates below 80% receive 0 points. At 80% the score is 0, at 100% the score is 30, with linear interpolation in between.

2.2 Phase 2: Cross-Evaluation Tournament (30% of project score)

Phase 2 is structured as a decentralized, all-to-all peer-to-peer cross-evaluation. The 30% allocation is divided equally between shooting and goalkeeping (15% each). After completing basic single-agent

policies in Phase 1, teams continue to improve their agents—through adversarial training, self-play, or further single-agent optimization—and then compete against every other team.

Because each team will design unique observation spaces, feature extractors, and neural network architectures, your codebases will be heterogeneous. To enable cross-evaluation without requiring teams to share code or checkpoints, the tournament adopts a standardized API interface: each team deploys their trained policy behind a lightweight server, and the competition script `scripts/compete.py` loads two G1 robots into a single MuJoCo scene and queries each team's server at every simulation step.

2.2.1 API Protocol

All teams must implement a policy server exposing the following two endpoints (a reference implementation is provided as `scripts/api_server.py`):

- `POST /act` — Receives an observation and returns an action.

```
Request:  {"observation": [[f32, ...]]}
Response: {"action": [[f32, ...]]}
```

- `POST /reset` — Resets the policy's internal state (LSTM hidden state, history buffer, etc.).

```
Request:  {}
Response: {"status": "ok"}
```

Teams may implement the server in any framework (the reference uses FastAPI), as long as the endpoints and JSON formats match exactly.

2.2.2 Timeline and Milestones

- **Weeks 13–16: Agent Development & Improvement**
  Teams first complete their basic single-agent shooting and goalkeeping policies (Phase 1). They then continuously improve their agents' capabilities through adversarial training, self-play, or further single-agent optimization. By the end of Week 16 (6.21), every team must freeze their code and publicly publish a GitHub repository containing their complete implementation, training configurations, and final trained checkpoints for both shooter and goalkeeper.
- **Week 17: Tournament & Report Submission**
  Teams deploy their policy servers and the tournament is conducted using `scripts/compete.py`. For each pair of teams (Team A and Team B), both directions are evaluated:
  - Team A's Shooter vs. Team B's Goalkeeper (10 trials)
  - Team B's Shooter vs. Team A's Goalkeeper (10 trials)

  Teams need to actively collaborate to cross-verify the env setup and the competition results. Both the tournament results and the final report are due by the end of Week 17 (6.26).

2.2.3 Scoring Mechanism

There are 14 teams in total. Each team's Phase 2 score is computed separately for shooting and goalkeeping (15 points maximum each):

- **Base Score:** Every team starts with 2 points for each task.
- **Head-to-Head Points:** For each opponent team, a direct matchup is played. Taking Team A's Shooter vs. Team B's Goalkeeper as an example: 10 trials are run. If Team A's shooter scores > 5 goals, Team A earns +1 point; otherwise (i.e., the goalkeeper holds the shooter to ≤ 5 goals), Team B earns +1 point. The same rule applies symmetrically to Team A's Goalkeeper vs. Team B's Shooter.

Each team plays every other team once per task direction, yielding 13 head-to-head opportunities per task. The maximum score per task is therefore $2 + 13 = 15$ points.

2.3 Submission and Deliverables (10% of project score)

Submission consists of a PDF report and the corresponding code repository before the end of Week 17 (6.26). The PDF report must include:

- **Phase 1 Results:** Basic shooting and goalkeeping evaluation results (success/block rates over 50 trials) with analysis.
- **Phase 2 Results:** Complete cross-evaluation results against all other teams—your shooter vs. each opponent's goalkeeper, and your goalkeeper vs. each opponent's shooter (10 trials each). Include a summary table of all matchups and the final tournament scores for both tasks.

3 Using the Template

The template is given at https://github.com/xiaojxkevin/G1-mjlab-soccer.

3.1 Repository Structure

The template provides:

- **Playable environments**: `Unitree-G1-Shooter` and `Unitree-G1-Goalkeeper` registered as mjlab tasks with configurable scene layout, ball physics, and MuJoCo visualization.
- **Evaluation scripts**: `eval_naive_shooter.py` and `eval_naive_goalkeeper.py` that load trained checkpoints, run headless batch trials, collect paper-matching metrics, and record videos. And you are allowed to modify these scripts if you want to use different observation spaces, reward functions etc.
- **Training configs**: Training environment designs under `config/training/` that specify observation spaces, reward functions, termination conditions, and domain randomization from the two papers. And it should be noticed that these are generated with coding agents and may contain errors. So you should carefully review and debug them before use.

3.2 Play and Visualization

Use the `scripts/play.py` script to visualize the environment:

```
# Shooter
python scripts/play.py Unitree-G1-Shooter --agent zero --viewer native

# Goalkeeper
python scripts/play.py Unitree-G1-Goalkeeper --agent zero --viewer native
```

3.3 Evaluation

Use the eval scripts to benchmark trained policies:

```
# Shooter
python scripts/eval_naive_shooter.py --headless --num-trials 50 \
    --checkpoint <path>

# Goalkeeper
python scripts/eval_naive_goalkeeper.py --headless --num-trials 50 \
    --checkpoint <path>
```

3.4 Implementing Training

The `train.py` script in `scripts/` serves as the training entrypoint. It is expected to register their own training tasks, implement the model classes, and run PPO training.

3.5 Cross-Evaluation Competition

For Phase 2 cross-evaluation, teams deploy their policies as API servers and use `scripts/compete.py` to run matchups:

```
# Start a policy server
python scripts/api_server.py --checkpoint <path> --port 8000 --task shooter

# Run a matchup (local checkpoint mode, for debugging)
python scripts/compete.py \
    --shooter-checkpoint <team_a_shooter.pt> \
    --goalkeeper-checkpoint <team_b_goalkeeper.pt> \
    --headless --num-trials 10

# Run a matchup (REST API mode, for tournament)
python scripts/compete.py \
    --shooter-api http://<team_a_ip>:8000 \
    --goalkeeper-api http://<team_b_ip>:8000 \
    --headless --num-trials 10
```

The script loads two G1 robots into a single MuJoCo scene, sends observations to each team's API (or local policy), concatenates their actions, and reports head-to-head statistics (goals scored, blocks). A reference API server implementation is provided as `scripts/api_server.py`.

3.6 Contributing and Reporting Issues

This template is a work in progress. The training configs, environment wrappers, and evaluation scripts were partially generated by coding agents and may contain bugs or rough edges. **We strongly encourage all students to**:

- **Report issues** — if you find a bug, a broken config, or unclear documentation, open a GitHub Issue on the template repository.
- **Submit pull requests** — fixes, improvements, and additional utilities (e.g., better observation configs, visualization tools, or multi-agent helpers) are welcome. PRs that benefit the whole class will be merged and acknowledged.

Treat this repository as a living codebase that improves with your feedback throughout the semester.

References

[1] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, "Learning soccer skills for humanoid robots: A progressive perception-action framework," 2026. [Online]. Available: https://arxiv.org/abs/2602.05310

[2] J. Ren, J. Long, T. Huang, H. Wang, Z. Wang, F. Jia, W. Zhang, J. Wang, P. Luo, and J. Pang, "Humanoid goalkeeper: Learning from position conditioned task-motion constraints," 2025.